PROJECT AND TEAM INFORMATION

Project Title

LexiScan: A Lexical Analyzer in C

Student / Team Information

Team Name: Team #	***CD-VI-T209***
Team member 1 (Team Lead)	*Bhandari, Amisha– 23022264* *bhandariamisha45@gmail.com*
Team member 2	*Sundriyal, Priyanshu – 23021752* *priyanshusundriyal65@gmail.com*

Team Name: *Team #*	***CD-VI-T209***
Team member 3	*Negi, Kartik – 23021080* *kartik24negi@gmail.com*
Team member 4	*Bist, Achintya – 23022358* *achintyabist05@gmail.com*

PROPOSAL DESCRIPTION

Motivation

Modern programming languages use compilers and interpreters to convert human-readable code into machine instructions. A key step in this process is lexical analysis, where the source code is divided into small units called tokens, such as keywords, identifiers, operators, and constants. This step is important because it prepares the code for the later stages of compilation.

The purpose of this project is to build a simple and easy-to-understand lexical analyzer using C[3].

The project helps illustrate how compilers read input, detect patterns, categorize tokens, and identify invalid symbols. Understanding this stage is important because lexical errors are often the first barrier between human code and executable programs.

State of the Art / Current solution

Today, lexical analysis is mostly done using compiler tools. Nowadays, compilers like GCC[5] use advanced analyzer tools such as Lex and Flex to convert source code into tokens.

These tools are difficult for beginners to understand. The working of these tools is hidden inside large systems, so students cannot clearly see how tokens are identified step by step.

Our project solves this problem by building a simple lexical analyzer in C[3]. It shows how tokenization, pattern matching, and error detection work.

Project Goals and Milestones

The aim of this project is to design and build a lexical analyzer in C[3] that takes source code as input and breaks it into tokens. This project shows how lexical analysis actually works inside a compiler in a simple and understandable way.

Our goal is to create a system that recognize keywords, identifiers, constants, operators and delimiters. It can also detect invalid tokens. The system will be able to detect lexical errors and will give proper messages.

Steps to complete this project:

1. First step is to analyze the requirements of C[3] language and the rules of tokenization.
2. Then we will build the primary tokenizer using C[3] which will handle the scanning of input characters and will break them into tokens.
3. Next we will then include an error module to identify and manage lexical errors during the scanning process.
4. The accuracy of the system will be checked using different test cases.
5. At last, we will prepare the documentation and optimization of code will be done.

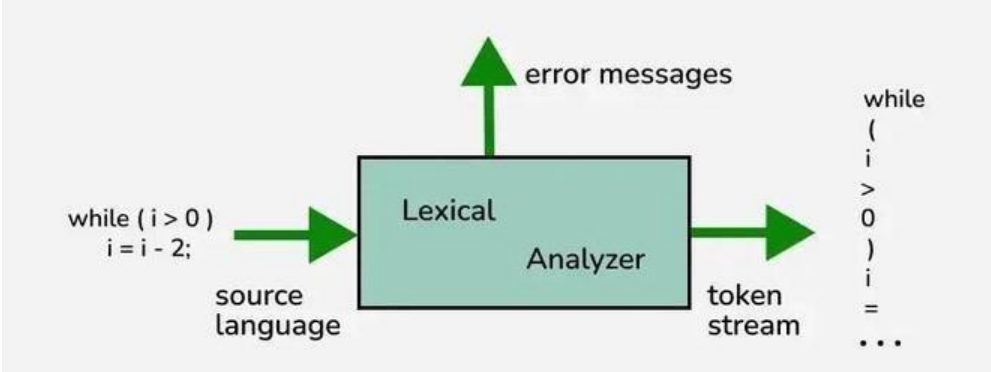
Project Approach

The process to build our project is divided into multiple steps. The rules are divided into small parts such as handling the input, recognizing tokens, classification, and detection of errors. The source code will be read from a file by the program and then the program will process the input character by character using pattern matching rules.

The project will be based on standard C[3] so it can run on different systems and allow direct control over memory and strings. Development will be done on Linux using GCC[5] and an editor like VS Code[2]. The analyzer will be tested using different sample programs to check accuracy.

The final system will focus on ease of understanding and learning the workings of a lexical analyzer inside a compiler.

System Architecture (High Level Diagram)



The system begins with a source code input module that reads characters from a file or user input. The input handler forwards the data to the lexical scanner, which performs pattern matching to identify token boundaries. The token classifier categorizes each token into keywords, identifiers, operators, constants, or symbols. An error detection module checks for illegal characters or malformed tokens. Finally, the processed tokens are sent to the output module for display or storage.

Project Outcome / Deliverables

The main outcome of this project will be a working lexical analyzer written in C[3] that can read source code and convert it into categorized tokens. The program will be able to recognize keywords, identifiers, operators, constants, delimiters, and invalid tokens, and it will display clear output along with error messages. The program will be modular, easy to read, and properly commented so that it can be understood.

Assumptions

This project assumes that the input program will be in C[3] language. The analyzer will only perform lexical analysis and will not check syntax or program meaning. It is also assumed that the input file is readable and not corrupted.

References

1. A. V. Aho, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques and Tools, 2nd ed. Noida, India: Pearson Education, 2013.
2. Microsoft, "Visual Studio Code Documentation," Microsoft Learn, 2024. [Online]. Available: https://code.visualstudio.com/docs
3. cppreference.com, "C Programming Language Reference," 2024. [Online]. Available: https://en.cppreference.com/w/c

[4]V. Pai T., A. Jayanthila Devi, and P. S. Aithal, "A systematic literature review of lexical analyzer implementation techniques in compiler design," Int. J. Appl. Eng. Manag. Letters, vol. 4, no. 2, pp. 285–301, Dec. 2020. [Online]. Available: https://www.researchgate.net/publication/360317780_A_Systematic_Literature_Review_of_Lexical_Analyzer_Implementation_Techniques_in_Compiler_Design

[5] GNU Project, "GCC Online Documentation," Free Software Foundation, 2024. [Online]. Available: https://gcc.gnu.org/onlinedocs/